Complete Java program: LinkedList methods

```
package com.company.collectionframework;

import java.util.*;

public class CollectionDemo {

    public static void main(String[] args) {

        LinkedList<String> list = new LinkedList<>();

        // 1. INSERTION METHODS
        System.out.println("===== 1. INSERTION METHODS =====");

        list.add("Rajiv");
        list.add("Sonal");
        list.add("Rohan");
        list.add("Rajiv"); // Duplicates are allowed

        list.add(1, "Riya"); // Insert at specified index
        list.addFirst("First");
        list.addLast("Last");

        System.out.println("add(), add(index, element), addFirst(),
addLast():");
        System.out.println(list);

        // 2. TRAVERSAL METHODS
        System.out.println("\n===== 2. TRAVERSAL METHODS =====");

        System.out.println("Using iterator():");
        Iterator<String> it1 = list.iterator();
        while (it1.hasNext()) {
            System.out.println(it1.next());
        }

        System.out.println("\nUsing descendingIterator():");
        Iterator<String> it2 = list.descendingIterator();
        while (it2.hasNext()) {
            System.out.println(it2.next());
        }

        System.out.println("\nUsing listIterator(2):");
        ListIterator<String> it3 = list.listIterator(2);
        while (it3.hasNext()) {
            System.out.println(it3.next());
        }

        System.out.println("\nUsing for-each loop:");
        for (String name : list) {
            System.out.println(name);
        }

        // 3. RETRIEVAL METHODS
```

```
        System.out.println("\n===== 3. RETRIEVAL METHODS =====");

        System.out.println("get(1): " + list.get(1));
        System.out.println("getFirst(): " + list.getFirst());
        System.out.println("getLast(): " + list.getLast());

        System.out.println("peek(): " + list.peek());
        System.out.println("peekFirst(): " + list.peekFirst());
        System.out.println("peekLast(): " + list.peekLast());

        // 4. SEARCHING METHODS
        System.out.println("\n===== 4. SEARCHING METHODS =====");

        System.out.println("contains(Rajiv): " + list.contains("Rajiv"));
        System.out.println("indexOf(Rajiv): " + list.indexOf("Rajiv"));
        System.out.println("lastIndexOf(Rajiv): " +
list.lastIndexOf("Rajiv"));
        System.out.println("size(): " + list.size());
        System.out.println("isEmpty(): " + list.isEmpty());

        // 5. QUEUE METHODS
        System.out.println("\n===== 5. QUEUE METHODS =====");

        list.offer("Ishika");
        System.out.println("After offer(): " + list);

        System.out.println("element(): " + list.element());
        System.out.println("peek(): " + list.peek());

        System.out.println("poll(): " + list.poll());
        System.out.println("After poll(): " + list);

        // 6. DEQUE METHODS
        System.out.println("\n===== 6. DEQUE METHODS =====");

        list.offerFirst("F1");
        list.offerLast("L1");
        System.out.println("After offerFirst() and offerLast(): " + list);

        System.out.println("removeFirst(): " + list.removeFirst());
        System.out.println("removeLast(): " + list.removeLast());

        list.addFirst("NewFirst");
        list.addLast("NewLast");
        System.out.println("After adding at both ends: " + list);

        // 7. STACK METHODS
        System.out.println("\n===== 7. STACK METHODS =====");

        list.push("PushElement");
        System.out.println("After push(): " + list);

        System.out.println("pop(): " + list.pop());
        System.out.println("After pop(): " + list);

        // 8. REMOVAL METHODS
        System.out.println("\n===== 8. REMOVAL METHODS =====");
```

```
        list.remove("Rajiv"); // Removes first matching occurrence
        System.out.println("After remove(Object): " + list);

        list.remove(0); // Removes element at index 0
        System.out.println("After remove(index): " + list);

        list.removeFirstOccurrence("Rohan");
        System.out.println("After removeFirstOccurrence(): " + list);

        list.removeLastOccurrence("Rajiv");
        System.out.println("After removeLastOccurrence(): " + list);

        // 9. CONDITIONAL REMOVAL
        System.out.println("\n===== 9. CONDITIONAL REMOVAL =====");

        list.removeIf(name -> name.startsWith("R"));
        System.out.println("After removeIf(): " + list);

        // 10. REPLACEMENT AND ARRAY CONVERSION
        System.out.println("\n===== 10. REPLACEMENT AND ARRAY CONVERSION
=====");

        if (!list.isEmpty()) {
            list.set(0, "UpdatedName");
        }
        System.out.println("After set(): " + list);

        Object[] arr = list.toArray();
        System.out.println("Array: " + Arrays.toString(arr));

        // 11. CLEARING THE LIST
        System.out.println("\n===== 11. CLEARING THE LIST =====");

        list.clear();
        System.out.println("After clear(): " + list);
        System.out.println("isEmpty(): " + list.isEmpty());
    }
}
```

Method groups covered

Category	Methods
Insertion	`add()`, `add(index, element)`, `addFirst()`, `addLast()`
Traversal	`iterator()`, `descendingIterator()`, `listIterator()`, for-each
Retrieval	`get()`, `getFirst()`, `getLast()`, `peek()`, `peekFirst()`, `peekLast()`
Searching	`contains()`, `indexOf()`, `lastIndexOf()`
Queue	`offer()`, `element()`, `poll()`
Deque	`offerFirst()`, `offerLast()`, `removeFirst()`, `removeLast()`
Stack	`push()`, `pop()`
Removal	`remove()`, `removeFirstOccurrence()`, `removeLastOccurrence()`,

	`removeIf()`
Replacement and conversion	`set()`, `toArray()`
General utilities	`size()`, `isEmpty()`, `clear()`

Important notes for students:

- `LinkedList` implements both `List` and `Deque`, so it can behave as a list, queue, deque, or stack.
- `listIterator(2)` starts at index 2 and traverses forward; `ListIterator` also supports `hasPrevious()` and `previous()` for backward traversal.
- `poll()` and `peek()` return `null` when the list is empty, whereas `removeFirst()`, `getFirst()`, `pop()`, and `element()` throw an exception when the required element is unavailable.
- `removeIf(name -> name.startsWith("R"))` removes every element whose name starts with `R`.
- `LinkedList` permits duplicate elements and preserves insertion order.

Complete Java Program: HashSet Methods

```
package com.company.collectionframework;

import java.util.*;

public class HashSetDemo {

    public static void main(String[] args) {

        HashSet<String> set = new HashSet<>();

        // 1. INSERTION METHODS
        System.out.println("===== 1. INSERTION METHODS =====");

        set.add("Rajiv");
        set.add("Sonal");
        set.add("Rohan");
        set.add("Rajiv"); // Duplicate element: not added
        set.add("Aman");

        System.out.println("After add(): " + set);

        // addAll() - Add elements from another collection
        HashSet<String> newSet = new HashSet<>();
        newSet.add("Riya");
        newSet.add("Neha");

        set.addAll(newSet);
        System.out.println("After addAll(): " + set);

        // 2. TRAVERSAL METHODS
        System.out.println("\n===== 2. TRAVERSAL METHODS =====");

        System.out.println("Using iterator():");
        Iterator<String> it = set.iterator();

        while (it.hasNext()) {
            System.out.println(it.next());
        }

        System.out.println("\nUsing for-each loop:");
        for (String name : set) {
            System.out.println(name);
        }

        // forEach() - Java 8+
        System.out.println("\nUsing forEach():");
        set.forEach(System.out::println);

        // 3. SEARCHING METHODS
        System.out.println("\n===== 3. SEARCHING METHODS =====");

        System.out.println("contains(Rajiv): " + set.contains("Rajiv"));
        System.out.println("contains(John): " + set.contains("John"));
```

```
        System.out.println("size(): " + set.size());
        System.out.println("isEmpty(): " + set.isEmpty());

        // 4. REMOVAL METHODS
        System.out.println("\n===== 4. REMOVAL METHODS =====");

        set.remove("Sonal");
        System.out.println("After remove(): " + set);

        // removeAll() - Remove all elements also present in newSet
        set.removeAll(newSet);
        System.out.println("After removeAll(): " + set);

        // 5. RETAINING COMMON ELEMENTS
        System.out.println("\n===== 5. retainAll() METHOD =====");

        HashSet<String> commonSet = new HashSet<>();
        commonSet.add("Rajiv");
        commonSet.add("Aman");
        commonSet.add("Karan");

        set.retainAll(commonSet);
        System.out.println("After retainAll(): " + set);

        // 6. CONDITIONAL REMOVAL
        System.out.println("\n===== 6. removeIf() METHOD =====");

        set.add("Rohan");
        set.add("Riya");
        set.add("Aman");

        set.removeIf(name -> name.startsWith("R"));
        System.out.println("After removeIf(): " + set);

        // 7. ARRAY CONVERSION
        System.out.println("\n===== 7. ARRAY CONVERSION =====");

        Object[] arr = set.toArray();
        System.out.println("Array: " + Arrays.toString(arr));

        String[] names = set.toArray(new String[0]);
        System.out.println("String array: " + Arrays.toString(names));

        // 8. CLONING
        System.out.println("\n===== 8. clone() METHOD =====");

        HashSet<String> copy = (HashSet<String>) set.clone();
        System.out.println("Original set: " + set);
        System.out.println("Cloned set: " + copy);

        // 9. CLEARING THE SET
        System.out.println("\n===== 9. clear() METHOD =====");

        set.clear();
        System.out.println("After clear(): " + set);
        System.out.println("isEmpty(): " + set.isEmpty());
    }
```

```
}
```

Methods grouped by category

Category	Methods
Insertion	`add()`, `addAll()`
Traversal	`iterator()`, `forEach()`, enhanced `for` loop
Searching	`contains()`, `size()`, `isEmpty()`
Removal	`remove()`, `removeAll()`
Retaining elements	`retainAll()`
Conditional removal	`removeIf()`
Conversion	`toArray()`
Copying	`clone()`
General utilities	`clear()`

Important points about HashSet

- No duplicates: Adding an existing element does not create another entry.
- No guaranteed order: Elements are not necessarily displayed in insertion or sorted order.
- Null support: A `HashSet` permits one `null` element.
- No index-based methods: Methods such as `get(0)` and `set(0, value)` are unavailable.
- Equality: Uniqueness is determined using `equals()` and `hashCode()`.
- Performance: `add()`, `remove()`, and `contains()` generally operate in \(O(1)\) average time.

Java Program for `LinkedHashSet` Class

The `LinkedHashSet` class in Java is part of the Java Collections Framework. It stores unique elements while maintaining their insertion order.

- Duplicates: Not allowed.
- Insertion order: Preserved.
- Null values: One `null` value is allowed.
- Underlying structure: Hash table and linked list.
- Performance: `add()`, `remove()`, and `contains()` generally take \(O(1)\) average time.

1. Java Program

```
package com.company.collectionframework;

import java.util.*;

public class LinkedHashSetDemo {

    public static void main(String[] args) {

        LinkedHashSet<String> set = new LinkedHashSet<>();

        // 1. INSERTION METHODS
        System.out.println("----- INSERTION METHODS -----");

        set.add("Rajiv");
        set.add("Sonal");
        set.add("Rohan");
        set.add("Rajiv");  // Duplicate: ignored
        set.add("Aman");
        set.add(null);     // Null is allowed
        set.add(null);     // Duplicate null: ignored

        System.out.println("After add(): " + set);

        LinkedHashSet<String> newSet = new LinkedHashSet<>();
        newSet.add("Riya");
        newSet.add("Neha");

        set.addAll(newSet);
        System.out.println("After addAll(): " + set);

        // 2. TRAVERSAL METHODS
        System.out.println("\n----- TRAVERSAL METHODS -----");

        System.out.println("Using Iterator:");
        Iterator<String> it = set.iterator();
        while (it.hasNext()) {
            System.out.println(it.next());
        }
```

```
System.out.println("Using enhanced for loop:");
for (String name : set) {
    System.out.println(name);
}

System.out.println("Using forEach():");
set.forEach(System.out::println);

// 3. SEARCHING AND SIZE METHODS
System.out.println("\n----- SEARCHING METHODS -----");

System.out.println("Contains Rajiv: " + set.contains("Rajiv"));
System.out.println("Contains John: " + set.contains("John"));
System.out.println("Size: " + set.size());
System.out.println("Is empty: " + set.isEmpty());

// 4. REMOVAL METHODS
System.out.println("\n----- REMOVAL METHODS -----");

set.remove("Sonal");
System.out.println("After remove(): " + set);

set.removeAll(newSet);
System.out.println("After removeAll(): " + set);

// 5. RETAINING COMMON ELEMENTS
System.out.println("\n----- RETAINING ELEMENTS -----");

LinkedHashSet<String> commonSet = new LinkedHashSet<>();
commonSet.add("Rajiv");
commonSet.add("Aman");
commonSet.add("Karan");

set.retainAll(commonSet);
System.out.println("After retainAll(): " + set);

// 6. CONDITIONAL REMOVAL
System.out.println("\n----- CONDITIONAL REMOVAL -----");

set.add("Rohan");
set.add("Riya");
set.add("Aman");

set.removeIf(name -> name != null && name.startsWith("R"));
System.out.println("After removeIf(): " + set);

// 7. CONVERSION TO ARRAYS
System.out.println("\n----- ARRAY CONVERSION -----");

Object[] arr = set.toArray();
System.out.println("Object array: " + Arrays.toString(arr));

String[] names = set.toArray(new String[0]);
System.out.println("String array: " + Arrays.toString(names));

// 8. CLONING / COPYING
System.out.println("\n----- COPYING -----");
```

```
        LinkedHashSet<String> copy = new LinkedHashSet<>(set);
        System.out.println("Original set: " + set);
        System.out.println("Copied set: " + copy);

        // 9. CLEARING
        System.out.println("\n----- CLEARING -----");

        set.clear();
        System.out.println("After clear(): " + set);
        System.out.println("Is empty: " + set.isEmpty());
    }
}
```

2. LinkedHashSet methods by category

Category	Methods	Purpose
Insertion	`add(E e)`	Adds a unique element
Insertion	`addAll(Collection c)`	Adds elements from another collection
Traversal	`iterator()`	Returns an iterator
Traversal	`forEach()`	Performs an action for each element
Searching	`contains(Object o)`	Checks whether an element exists
Information	`size()`	Returns the number of elements
Information	`isEmpty()`	Checks whether the set is empty
Removal	`remove(Object o)`	Removes a specified element
Removal	`removeAll(Collection c)`	Removes all elements found in another collection
Retention	`retainAll(Collection c)`	Keeps only common elements
Conditional removal	`removeIf(Predicate p)`	Removes elements satisfying a condition
Conversion	`toArray()`	Converts the set into an array
Copying	`new LinkedHashSet<>(set)`	Creates a separate set containing the same elements
Clearing	`clear()`	Removes all elements

3. Difference between `HashSet`, `LinkedHashSet`, and `TreeSet`

Feature	`HashSet`	`LinkedHashSet`	`TreeSet`
Duplicates	Not allowed	Not allowed	Not allowed
Order	No guaranteed order	Insertion order	Sorted order by default
Null	Allows one `null`	Allows one `null`	Generally does not allow `null` with natural ordering

Typical performance	\(O(1)\) average	\(O(1)\) average	\(O(\log n)\)
Best suited for	Fast uniqueness checks	Uniqueness with insertion order	Sorted unique elements

4. Example: Understanding insertion order

```
LinkedHashSet<Integer> numbers = new LinkedHashSet<>();

numbers.add(30);
numbers.add(10);
numbers.add(20);
numbers.add(10);

System.out.println(numbers);
```

Output:

```
[30, 10, 20]
```

The second occurrence of `10` is ignored, and the original insertion order is preserved.

Important: `LinkedHashSet` does not provide index-based methods such as `get(0)` because it implements `Set`, not `List`. It preserves insertion order, but it is not a sorted collection.

Complete Java Program for `LinkedHashSet` Class

The `LinkedHashSet` class in Java is part of the Java Collections Framework. It stores unique elements while maintaining their insertion order.

- Duplicates: Not allowed.
- Insertion order: Preserved.
- Null values: One `null` value is allowed.
- Underlying structure: Hash table and linked list.
- Performance: `add()`, `remove()`, and `contains()` generally take \(O(1)\) average time.

1. Complete Java Program

```
package com.company.collectionframework;

import java.util.*;

public class LinkedHashSetDemo {

    public static void main(String[] args) {
```

```
LinkedHashSet<String> set = new LinkedHashSet<>();

// 1. INSERTION METHODS
System.out.println("----- INSERTION METHODS -----");

set.add("Rajiv");
set.add("Sonal");
set.add("Rohan");
set.add("Rajiv");  // Duplicate: ignored
set.add("Aman");
set.add(null);     // Null is allowed
set.add(null);     // Duplicate null: ignored

System.out.println("After add(): " + set);

LinkedHashSet<String> newSet = new LinkedHashSet<>();
newSet.add("Riya");
newSet.add("Neha");

set.addAll(newSet);
System.out.println("After addAll(): " + set);

// 2. TRAVERSAL METHODS
System.out.println("\n----- TRAVERSAL METHODS -----");

System.out.println("Using Iterator:");
Iterator<String> it = set.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}

System.out.println("Using enhanced for loop:");
for (String name : set) {
    System.out.println(name);
}

System.out.println("Using forEach():");
set.forEach(System.out::println);

// 3. SEARCHING AND SIZE METHODS
System.out.println("\n----- SEARCHING METHODS -----");

System.out.println("Contains Rajiv: " + set.contains("Rajiv"));
System.out.println("Contains John: " + set.contains("John"));
System.out.println("Size: " + set.size());
System.out.println("Is empty: " + set.isEmpty());

// 4. REMOVAL METHODS
System.out.println("\n----- REMOVAL METHODS -----");

set.remove("Sonal");
System.out.println("After remove(): " + set);

set.removeAll(newSet);
System.out.println("After removeAll(): " + set);

// 5. RETAINING COMMON ELEMENTS
```

```
        System.out.println("\n----- RETAINING ELEMENTS -----");

        LinkedHashSet<String> commonSet = new LinkedHashSet<>();
        commonSet.add("Rajiv");
        commonSet.add("Aman");
        commonSet.add("Karan");

        set.retainAll(commonSet);
        System.out.println("After retainAll(): " + set);

        // 6. CONDITIONAL REMOVAL
        System.out.println("\n----- CONDITIONAL REMOVAL -----");

        set.add("Rohan");
        set.add("Riya");
        set.add("Aman");

        set.removeIf(name -> name != null && name.startsWith("R"));
        System.out.println("After removeIf(): " + set);

        // 7. CONVERSION TO ARRAYS
        System.out.println("\n----- ARRAY CONVERSION -----");

        Object[] arr = set.toArray();
        System.out.println("Object array: " + Arrays.toString(arr));

        String[] names = set.toArray(new String[0]);
        System.out.println("String array: " + Arrays.toString(names));

        // 8. CLONING / COPYING
        System.out.println("\n----- COPYING -----");

        LinkedHashSet<String> copy = new LinkedHashSet<>(set);
        System.out.println("Original set: " + set);
        System.out.println("Copied set: " + copy);

        // 9. CLEARING
        System.out.println("\n----- CLEARING -----");

        set.clear();
        System.out.println("After clear(): " + set);
        System.out.println("Is empty: " + set.isEmpty());
    }
}
```

2. LinkedHashSet methods by category

Category	Methods	Purpose
Insertion	`add(E e)`	Adds a unique element
Insertion	`addAll(Collection c)`	Adds elements from another collection
Traversal	`iterator()`	Returns an iterator
Traversal	`forEach()`	Performs an action for each element

Searching	`contains(Object o)`	Checks whether an element exists
Information	`size()`	Returns the number of elements
Information	`isEmpty()`	Checks whether the set is empty
Removal	`remove(Object o)`	Removes a specified element
Removal	`removeAll(Collection c)`	Removes all elements found in another collection
Retention	`retainAll(Collection c)`	Keeps only common elements
Conditional removal	`removeIf(Predicate p)`	Removes elements satisfying a condition
Conversion	`toArray()`	Converts the set into an array
Copying	`new LinkedHashSet<>(set)`	Creates a separate set containing the same elements
Clearing	`clear()`	Removes all elements

3. Difference between `HashSet`, `LinkedHashSet`, and `TreeSet`

Feature	`HashSet`	`LinkedHashSet`	`TreeSet`
Duplicates	Not allowed	Not allowed	Not allowed
Order	No guaranteed order	Insertion order	Sorted order by default
Null	Allows one `null`	Allows one `null`	Generally does not allow `null` with natural ordering
Typical performance	\(O(1)\) average	\(O(1)\) average	\(O(\log n)\)
Best suited for	Fast uniqueness checks	Uniqueness with insertion order	Sorted unique elements

4. Example: Understanding insertion order

```
LinkedHashSet<Integer> numbers = new LinkedHashSet<>();

numbers.add(30);
numbers.add(10);
numbers.add(20);
numbers.add(10);

System.out.println(numbers);
```

Output:

```
[30, 10, 20]
```

The second occurrence of `10` is ignored, and the original insertion order is preserved.

Important: `LinkedHashSet` does not provide index-based methods such as `get(0)` because it implements `Set`, not `List`. It preserves insertion order, but it is not a sorted collection.

Java Program for `TreeSet` Class

The `TreeSet` class is part of the Java Collections Framework. It stores unique elements in sorted order and implements the `NavigableSet` interface.

- Duplicates: Not allowed.
- Ordering: Ascending order by default.
- Custom ordering: Supported through a `Comparator`.
- Null values: Generally not allowed with natural ordering.
- Performance: Basic operations such as `add()`, `remove()`, and `contains()` take \(O(\log n)\) time.

1. Java program

This program demonstrates the major `TreeSet` methods, grouped by category.

```
package com.company.collectionframework;

import java.util.*;

public class TreeSetDemo {

    public static void main(String[] args) {

        TreeSet<Integer> set = new TreeSet<>();

        // 1. INSERTION METHODS
        System.out.println("----- INSERTION METHODS -----");

        set.add(40);
        set.add(10);
        set.add(30);
        set.add(20);
        set.add(10); // Duplicate ignored

        System.out.println("After add(): " + set);

        TreeSet<Integer> anotherSet = new TreeSet<>();
        anotherSet.add(50);
        anotherSet.add(60);

        set.addAll(anotherSet);
        System.out.println("After addAll(): " + set);

        // 2. TRAVERSAL METHODS
        System.out.println("\n----- TRAVERSAL METHODS -----");

        System.out.println("Using Iterator:");
        Iterator<Integer> it = set.iterator();
        while (it.hasNext()) {
            System.out.println(it.next());
```

```
}

System.out.println("Using descendingIterator:");
Iterator<Integer> reverseIt = set.descendingIterator();
while (reverseIt.hasNext()) {
    System.out.println(reverseIt.next());
}

System.out.println("Using enhanced for loop:");
for (Integer value : set) {
    System.out.println(value);
}

// 3. SEARCHING AND SIZE METHODS
System.out.println("\n----- SEARCHING METHODS -----");

System.out.println("Contains 30: " + set.contains(30));
System.out.println("Contains 90: " + set.contains(90));
System.out.println("Size: " + set.size());
System.out.println("Is empty: " + set.isEmpty());

// 4. FIRST, LAST AND NAVIGATION METHODS
System.out.println("\n----- NAVIGATION METHODS -----");

System.out.println("First: " + set.first());
System.out.println("Last: " + set.last());

System.out.println("Lower than 30: " + set.lower(30));
System.out.println("Floor of 30: " + set.floor(30));
System.out.println("Ceiling of 35: " + set.ceiling(35));
System.out.println("Higher than 30: " + set.higher(30));

// 5. SUBSET METHODS
System.out.println("\n----- SUBSET METHODS -----");

System.out.println("headSet(30): " + set.headSet(30));
System.out.println("headSet(30, true): " + set.headSet(30, true));

System.out.println("tailSet(30): " + set.tailSet(30));
System.out.println("tailSet(30, false): " + set.tailSet(30, false));

System.out.println("subSet(20, 50): "
        + set.subSet(20, 50));

System.out.println("subSet(20, true, 50, true): "
        + set.subSet(20, true, 50, true));

// 6. REMOVAL METHODS
System.out.println("\n----- REMOVAL METHODS -----");

set.remove(20);
System.out.println("After remove(20): " + set);

System.out.println("pollFirst(): " + set.pollFirst());
System.out.println("After pollFirst(): " + set);

System.out.println("pollLast(): " + set.pollLast());
```

```
        System.out.println("After pollLast(): " + set);

        set.removeAll(anotherSet);
        System.out.println("After removeAll(): " + set);

        // 7. CONDITIONAL REMOVAL
        System.out.println("\n----- CONDITIONAL REMOVAL -----");

        set.removeIf(value -> value > 30);
        System.out.println("After removeIf(value > 30): " + set);

        // 8. DESCENDING SET
        System.out.println("\n----- DESCENDING SET -----");

        System.out.println("Descending set: " + set.descendingSet());

        // 9. CLONING / COPYING
        System.out.println("\n----- COPYING -----");

        TreeSet<Integer> copy = new TreeSet<>(set);
        System.out.println("Original set: " + set);
        System.out.println("Copied set: " + copy);

        // 10. CLEARING
        System.out.println("\n----- CLEARING -----");

        set.clear();
        System.out.println("After clear(): " + set);
        System.out.println("Is empty: " + set.isEmpty());
    }
}
```

2. TreeSet methods by category

Category	Methods	Purpose
Insertion	`add(E e)`	Adds an element in sorted position
Insertion	`addAll(Collection c)`	Adds elements from another collection
Traversal	`iterator()`	Traverses in ascending order
Traversal	`descendingIterator()`	Traverses in descending order
Searching	`contains(Object o)`	Checks whether an element exists
Information	`size()`, `isEmpty()`	Returns size or checks emptiness
End elements	`first()`, `last()`	Returns smallest or largest element
Navigation	`lower(E e)`	Greatest element strictly less than `e`
Navigation	`floor(E e)`	Greatest element less than or equal to `e`
Navigation	`ceiling(E e)`	Smallest element greater than or equal to `e`
Navigation	`higher(E e)`	Smallest element strictly greater than `e`
Range views	`headSet()`, `tailSet()`, `subSet()`	Retrieves a sorted range view

Removal	`remove(), removeAll()`	Removes elements
End removal	`pollFirst(), pollLast()`	Removes and returns smallest or largest element
Conditional removal	`removeIf(Predicate p)`	Removes elements matching a condition
Ordering	`descendingSet()`	Returns a reverse-order view
Copying	`new TreeSet<>(set)`	Creates a separate set with the same elements
Clearing	`clear()`	Removes all elements

3. Understand navigation methods

Suppose the set contains:

```
TreeSet<Integer> numbers =
        new TreeSet<>(Arrays.asList(10, 20, 30, 40, 50));
```

Navigation example: target = 30

10

20

30

40

50

`lower(30)`

20

Strictly smaller

`floor(30)`

30

Smaller or equal

`ceiling(30)`

30

Greater or equal

```
higher(30)
```

40

Strictly greater

4. Custom sorting using `Comparator`

By default, `TreeSet` sorts numbers in ascending order and strings in natural alphabetical order. You can provide a comparator for a different order.

```
import java.util.*;

public class CustomTreeSetDemo {
    public static void main(String[] args) {

        // Descending order
        TreeSet<Integer> numbers =
                new TreeSet<>(Comparator.reverseOrder());

        numbers.add(10);
        numbers.add(40);
        numbers.add(20);
        numbers.add(30);

        System.out.println(numbers);
    }
}
```

Output:

```
[40, 30, 20, 10]
```

Sorting custom objects

```
import java.util.*;

class Student {
    int rollNo;
    String name;
    int marks;

    Student(int rollNo, String name, int marks) {
        this.rollNo = rollNo;
        this.name = name;
        this.marks = marks;
    }

    @Override
    public String toString() {
```

```
        return rollNo + " " + name + " " + marks;
    }
}

public class StudentTreeSetDemo {
    public static void main(String[] args) {

        TreeSet<Student> students = new TreeSet<>(
            Comparator.comparingInt((Student s) -> s.marks)
                      .reversed()
                      .thenComparingInt(s -> s.rollNo)
        );

        students.add(new Student(3, "Rohan", 85));
        students.add(new Student(1, "Aman", 95));
        students.add(new Student(2, "Sonal", 85));

        for (Student s : students) {
            System.out.println(s);
        }
    }
}
```

Output:

```
1 Aman 95
2 Sonal 85
3 Rohan 85
```

Here, students are sorted by marks in descending order, then by roll number in ascending order when marks are equal.

Important: A `TreeSet` considers two elements equivalent when its comparator returns `0`. If you want to retain students with equal marks, include a tie-breaker such as roll number. Also, `headSet()`, `tailSet()`, and `subSet()` return backed views, not independent copies.